CS651 Project 2 Proposal

Roomwise: A Photo and Voice Guided Room Planner

Team: Sean, Priyansh, Shubham, Linn **Date:** October 11, 2026 **Status:** Proposed design for team review

1. Application purpose and description

Roomwise will help users rearrange the furniture they already own using photos of their room and a spoken description of their goals. We will extend our Project 1 React room planner into a complete Express, React, and Node.js application. Users will select photos from their Google Photos account, record a short voice brief, confirm room and furniture measurements, and receive an editable room plan with explanations and a prioritized action list.

For example, a user might say, "Keep my sofa and desk, make more room near the door, and give me a quieter work corner." Gemini will analyze the selected room photos and the recorded audio. A Google ADK workflow will combine the visual observations and spoken requirements, produce a proposed arrangement, and check it against the user's confirmed measurements. The user can compare the original and suggested layouts, make changes, and save the result for later sessions.

Roomwise will use still photos and short voice recordings only. The application will not record, upload, extract frames from, or analyze video.

The output will include a furniture inventory, an editable two-dimensional layout, reasons for the suggested changes, constraint warnings, and a short action checklist. The application will emphasize rearrangement rather than product shopping. It will not promise exact measurements from photos, photorealistic rendering, or guaranteed accessibility compliance. Users will confirm dimensions and actual clearances before moving furniture.

1.1 Authentication and social photo access

We propose Google sign-in through Firebase Authentication for Roomwise accounts. The Express backend will verify Firebase ID tokens and associate each saved project with the verified user's UID. The existing guest sign-in interface will be extended to support real authentication.

Photo access will be a separate, explicit Google OAuth 2.0 consent flow using the Google Photos Picker API. After authorization, the backend will create a picker session. The user will follow the picker link, choose room photos in Google Photos, and return to Roomwise. The backend will poll the session as directed by the API, retrieve the selected media, and download the selected images for analysis. We will not assume unrestricted access to the user's entire photo library or permanent availability of picker media URLs.

The team will configure the OAuth consent screen, allowed callback URLs, and necessary scopes, and will test access with at least two independent accounts. Instructor access is a required acceptance criterion. Any test-user enrollment or verification needed for the selected scopes must be resolved before the demonstration. We will retain the AWS-hosted Project 1 website as the entry point and add a link to the Project 2 application. Any provider registration information or business website requested will be supplied there.

1.2 Gemini processing of photos and audio

Photo analysis: The Node backend will send user-selected photos to a Gemini model that supports image input. It will request structured observations: furniture categories, apparent relative placement, visible doors and windows, visual style, possible obstructions, and uncertain or missing information. Photo observations will be linked to their source images. Estimated geometry will remain provisional until the user confirms dimensions and positions.

Audio analysis: The React application will capture a fresh recording with the browser microphone, offer playback and re-recording, and upload the audio to Express. Gemini will process the audio itself, rather than receiving only a browser-generated transcript. We will request a transcript and structured requirements such as room use, furniture to retain, preferred changes, priorities, and any stated measurements. Users will review and correct the extracted brief before planning.

Both inputs affect the same decision. Photos describe the existing room; audio describes what the user wants to change. For example, identifying a desk in a photo becomes useful when the audio asks for a separate work area. Changing the photos or spoken brief will create a new analysis and plan version. The backend will use the current Google GenAI SDK and a supported multimodal Gemini model selected and tested during implementation. API credentials will remain on the server.

1.3 Google ADK agent architecture

Initial Gemini media calls will produce observations, not the final room plan. We propose a small Google ADK workflow deployed with the Node backend on Cloud Run. The implementation will use the supported ADK JavaScript/TypeScript package, with the selected package version pinned.

1. **Requirements Agent:** Combines photo observations, corrected audio requirements, confirmed measurements, and an existing saved plan when available. It produces a structured planning brief and asks for missing measurements or clarification when needed.

2. **Layout Agent:** Uses the brief to propose positions and rotations for the retained furniture. It invokes a deterministic `validateLayout` tool that checks room bounds, furniture overlap, and the user-confirmed door exclusion area. It may revise a failed candidate once; remaining conflicts are displayed rather than hidden.

3. **Explanation Agent:** Turns the candidate and validation results into reasons, warnings, and prioritized actions. It connects recommendations to the user's stated goals and flags unresolved assumptions.

An ADK workflow controller will run these agents in order and pass their structured outputs through session state. Supporting tools will include `getRoomProject` for an authorized read of the user's existing project and `validateLayout` for geometry checks. Project ownership will be checked by backend code; the model will not select a different user's UID. Saving a plan will require an explicit user action through the application API.

This architecture gives ADK a meaningful role: it transforms media observations into a personalized plan, uses validation tools, and explains the result. The validator extends Project 1's existing geometry checks; it does not establish real-world fit or walking clearance from photographs alone.

1.4 Firestore storage and retrieval

We will use Cloud Firestore with the proposed structure:

Collection/document	Proposed fields and purpose
`users/{uid}`	Display name and account timestamps; no passwords or API keys.
`users/{uid}/projects/{projectId}`	Room name, confirmed dimensions, door/window geometry, furniture inventory, original layout, and timestamps.
`.../projects/{projectId}/analyses/{analysisId}`	Photo references/metadata, structured visual observations, audio transcript, corrected requirements, model identifier, and status.
`.../projects/{projectId}/plans/{planId}`	Analysis reference, furniture positions/rotations, explanations, validation warnings, action list, and version timestamps.

The backend will create projects, list the signed-in user's projects, retrieve a selected project, update measurements or a saved layout, and delete a project with its child documents. Plan history will be queried by project and creation time. All routes will verify ownership.

Firestore will store structured results rather than raw media. If source media must be retained to support later review, it will be held in private Google Cloud Storage and referenced by object ID; otherwise it will be deleted after processing. Short-lived picker URLs will not serve as permanent image storage. Logs and analytics will exclude raw recordings, photo contents, OAuth tokens, and API credentials.

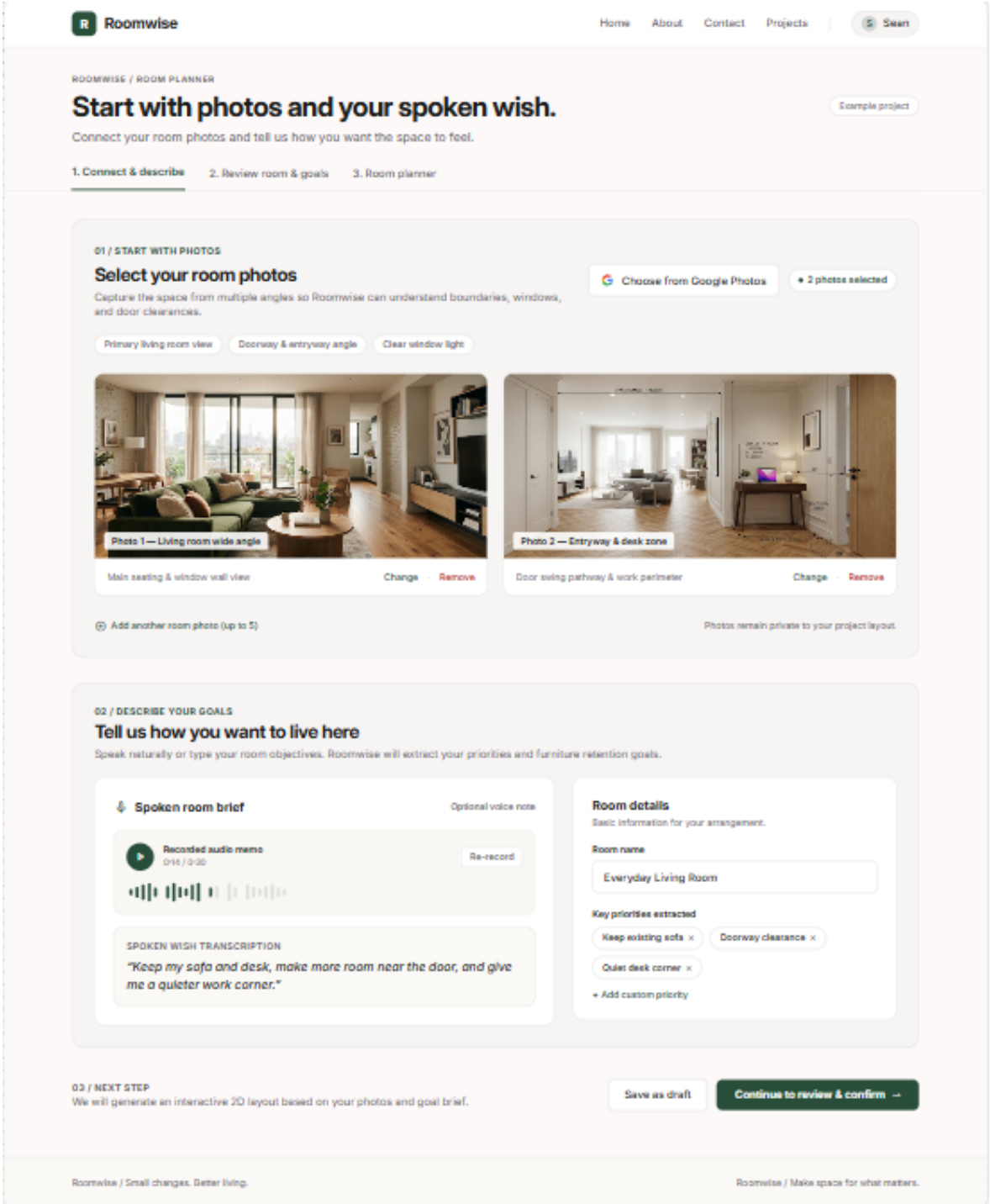
1.5 Evolution of the Project 1 frontend

Our existing source includes `src/workspace.jsx`, `src/login.jsx`, and `src/components.jsx`. We will extend this code rather than recreate the frontend.

Existing component or function	Proposed reuse or extension
`Navigation`, `Footer`, `FormField`	Retain shared navigation, branding, and form patterns; add authenticated state and new workspace links.
`RoomCanvas`, `FurnitureItem`	Retain SVG rendering, dragging, selection, keyboard movement, and furniture rotation; load user-specific inventory and layouts.
`NumericField`, `MeasurementPanel`	Retain dimension editing and scale confirmation; extend to user-confirmed geometry.
`checkLayout`	Extend bounds/overlap checks and replace the example room's fixed door assumptions with stored user geometry.
`SavedLayout`	Replace in-memory saving with Firestore-backed project and plan history.
`LoginPage` and workspace `App`	Replace guest-only behavior with verified sign-in and integrate photo, audio, analysis, and project APIs.

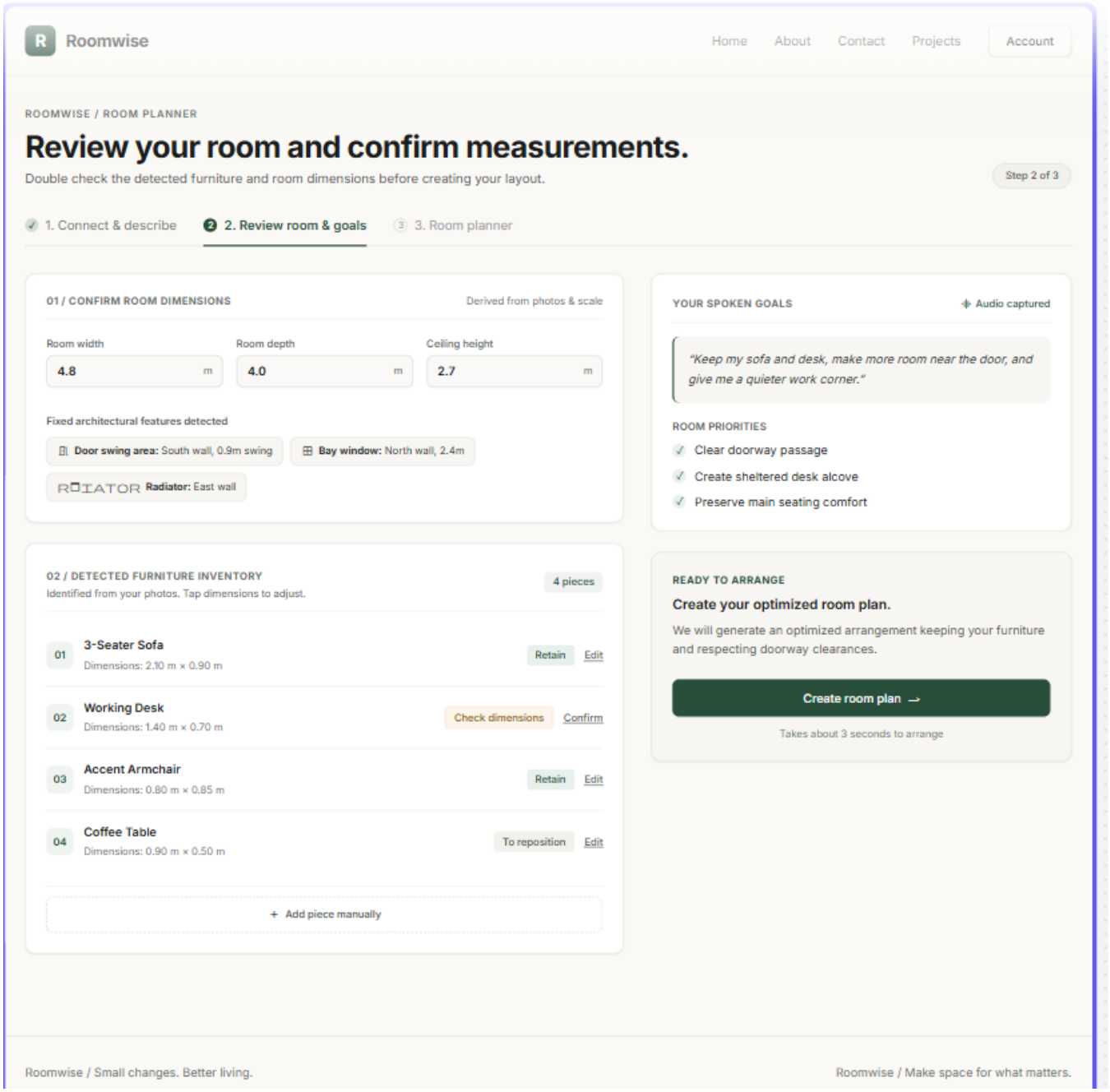
New React components will include `PhotoImportPanel`, `VoiceBriefRecorder`, `AnalysisReviewPanel`, `PlanningStatus`, `RecommendationPanel`, and `ProjectHistory`. Names are proposed implementation names. The editor will retain its existing layout while gaining an input stage and a results panel. React will call authenticated Express endpoints; it will not call Gemini with a secret key.

2. Mockup interfaces / Screen A - Connect and describe



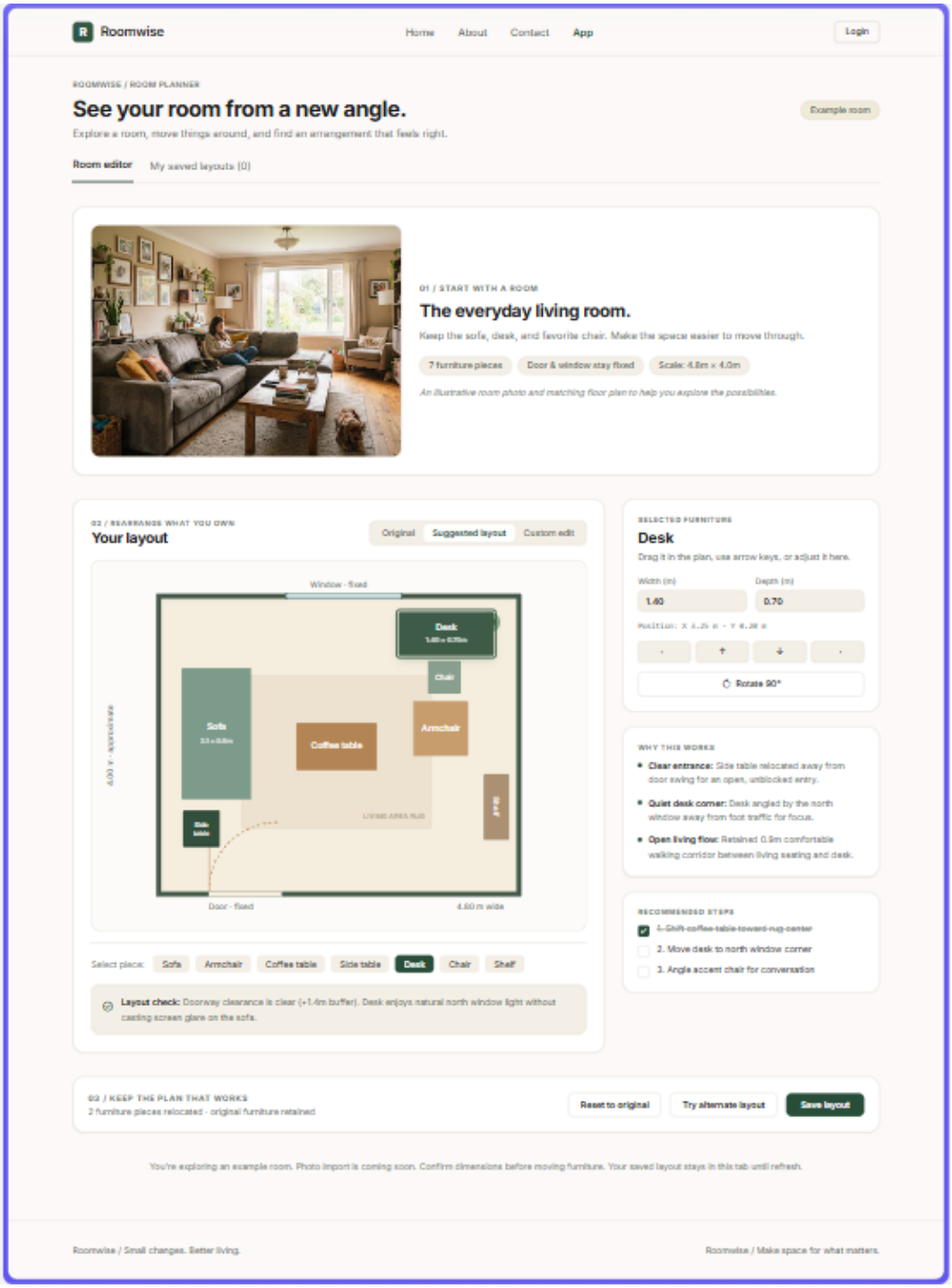
Users select room photos through Google Photos and record a fresh audio brief. The mockup labels voice input optional; the implemented analysis workflow will require photos and audio. No video recording or processing is planned.

2. Mockup interfaces / Screen B - Review and confirm



Users review detected furniture and spoken goals, correct errors, and confirm dimensions and fixed geometry. Values are illustrative estimates requiring confirmation. The displayed three-second processing time is a placeholder, not a performance promise.

2. Mockup interfaces / Screen C - Edit and save



The existing room editor is extended with agent recommendations, reasons, and validation warnings. Users edit furniture positions and save plans to Firestore. Mockup text about photo import coming soon and saving only in this tab will be replaced by authenticated photo import and persistent project history.

2. Mockup interfaces

The following Stitch mockups describe proposed behavior, not an implemented application. The accompanying PDF includes one supplied screenshot per screen.

Mockup labels will be aligned with implementation: audio is required for the full analysis workflow, the three-second processing time is a placeholder, photo import will be active, and saved layouts will persist in Firestore rather than only in the current tab. Dimensions and clearances shown are illustrative and require user confirmation.

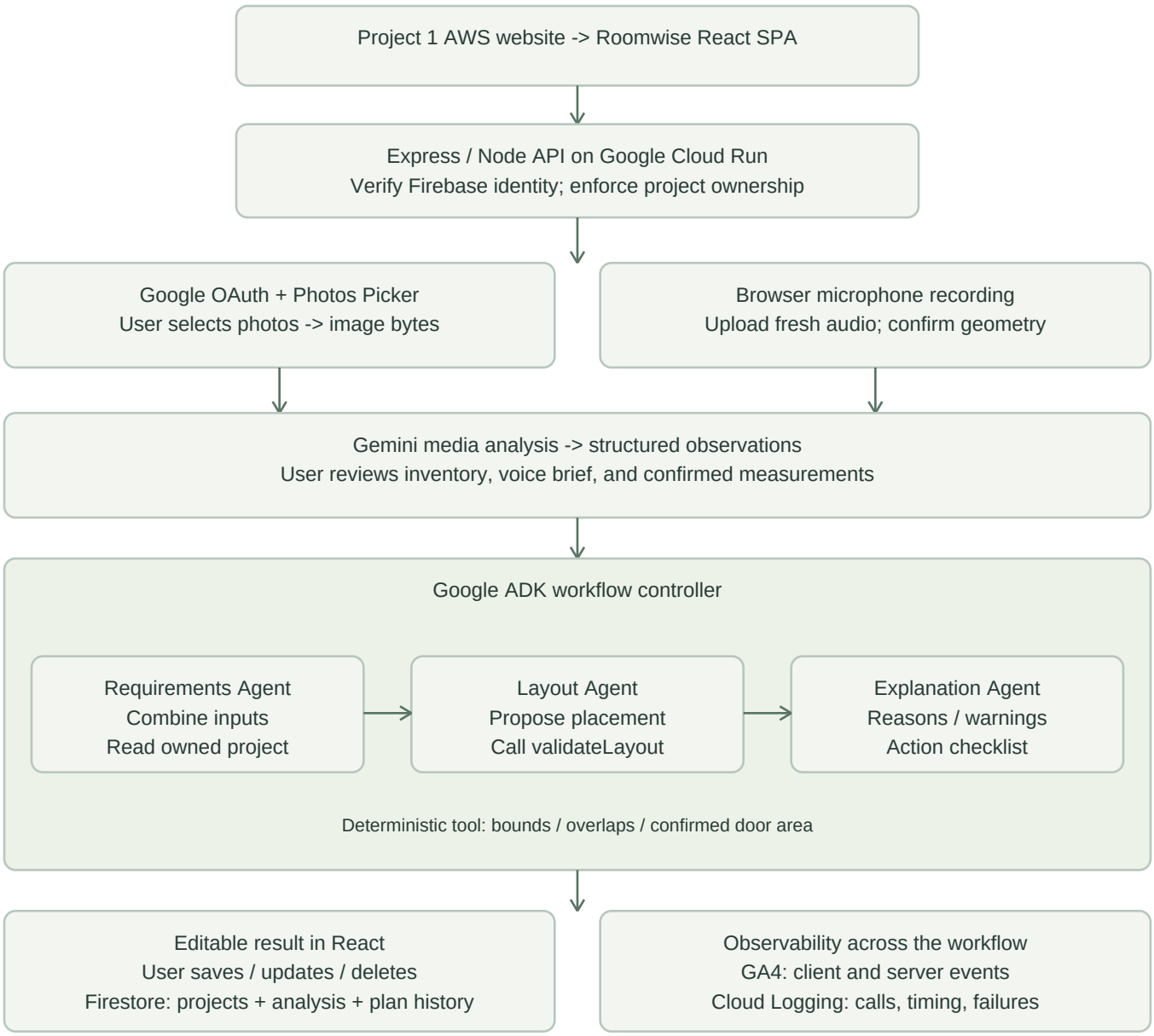
Screen A - Connect and describe: Roomwise navigation, signed-in account, Google Photos connect/select controls, selected photo thumbnails, microphone recording and playback controls, room name, and a Continue button. Photo selection and a fresh recording are required for the full analysis demonstration. Permission denial and picker timeouts will have recoverable messages.

Screen B - Review and confirm: Selected photos beside a furniture inventory and the editable voice brief. The user confirms room dimensions, furniture sizes, and door/window positions. Uncertain observations are highlighted. Generate plan becomes available when the minimum geometry is confirmed. A status display shows media analysis, agent planning, and validation.

Screen C - Edit and save: Original/suggested layout switch, the reused draggable room canvas, measurement controls, recommendation explanations, validation warnings, and an action checklist. Save plan, Update plan, and Project history controls support persistent storage and retrieval. Saved projects can be reopened after signing out and back in; deletion requires an explicit user action.

An illustrative result might explain: "Move the side table away from the confirmed door area and rotate the desk toward the window to support your requested work corner." This is an example of intended output, not a measured or generated result.

3. System diagram and agent architecture



Backend mediates photo access, Gemini calls, ADK execution, persistence, and logging. Initial media analysis precedes the agent workflow. Firebase sign-in and Photos OAuth have separate purposes.

4. Flow of control and technical details

1. The user follows the Project 1 website link to the HTTPS Cloud Run application and signs into Roomwise.

2. React obtains a Firebase ID token. Express verifies it before allowing access to project or analysis routes.

3. The user connects Google Photos through a separate OAuth consent flow and selects photos in a picker session. Express retrieves selected images after selection is complete.

4. React records the user's current voice brief, offers playback, and uploads the recording. Express checks media types and size/duration limits and records processing status.

5. Express calls Gemini for image observations and audio understanding and validates the returned structures. The app displays the extracted inventory and brief for correction.

6. The user confirms measurements and geometry. Express starts the ADK workflow with those confirmed values and the reviewed analysis.

7. The Requirements Agent builds the brief. The Layout Agent proposes a layout and invokes validation. The Explanation Agent produces grounded reasons and actions. Invalid or incomplete results are flagged for review.

8. Express returns a structured plan; React renders it through the reused room canvas. The user compares layouts and edits positions or dimensions.

9. On Save, Express writes the project, analysis, and plan to Firestore. History routes retrieve saved versions; update and delete routes support the full data lifecycle.

10. Client/server GA4 events and Cloud Logging capture feature usage, API outcomes, and timing. Failure states support retry without silently presenting partial work as complete.

Proposed API groups: `/api/projects` for owned project CRUD; `/api/photos/sessions` for picker sessions; `/api/analyses` for media processing; `/api/plans` for agent generation and saved plan operations. Long processing jobs will expose status so the interface can show progress. Frontend and backend code will be separated into directories as required.

5. Analytics, testing, and documentation

We will implement GA4 client events for photo selection, recording completion, plan requests, edits, and saves. Server-side Measurement Protocol events will capture successful and failed analysis/plan operations with the required configuration. Google Cloud Logging will record social-photo and Gemini API calls, ADK workflow stages, durations, and failures. Relevant frontend errors will be sent to an authenticated backend logging endpoint.

Our report will analyze at least three metrics: plan-generation completion rate, generated-plan save rate, and repeat project usage. We will define denominators and observation periods, present graphs, explain trends, and discuss limitations such as small samples and incomplete tracking. We will invite friends, classmates, and family to use the deployed app early enough to collect data.

Testing will cover at least two independent users, instructor-access preparation, addition and selection of a new Google Photos image, changed live audio requirements, changed output, Firestore create/read/update/delete behavior, persistent history, agent/tool execution, invalid geometry, permission denial, and API failures. Browser testing will follow the instructor's clarified browser requirements.

The GitHub wiki will use the required titles and include the proposal PDF, component descriptions, commit evidence, five issue-board screenshots collected over time, database evidence, Gemini/ADK evidence, analytics/logging report, presentation PDF, and complete unlisted YouTube demonstration. Each component/code file will have a corresponding GitHub issue. The required YouTube demonstration is a separate recording of the team presenting and demonstrating the completed application; video capture is not a Roomwise feature. The team will use consistent camelCase and descriptive code comments.

6. Proposed implementation schedule and scope

Period	Milestone
Oct. 10-13	Finalize proposal, wireframes, system diagram, and repository/wiki.
Oct. 14-23	Prove Google Photos access with multiple accounts; add real authentication, Express, and initial Cloud Run deployment.
Oct. 24-Nov. 6	Complete photo/audio Gemini analysis, user review, Firestore project CRUD, and input screens.
Nov. 7-16	Integrate ADK agents and geometry tools with the existing room editor; complete persistent plan history.
Nov. 17-23	Run user testing, collect analytics, fix integration problems, and gather demonstration evidence.
Nov. 24-30	Finish reports, wiki, presentation, browser checks, and complete demonstration recording.

Responsibilities will be agreed by the team before submission. Suggested work areas are React/editor integration, authentication/photo import, Gemini/audio/ADK, and Firestore/deployment/analytics; all members will review integration and participate in testing and presentation.

The core scope is one room per project, selected room photos, one short current voice brief per analysis, confirmed geometry, an editable plan, and persistent versions. Video recording or processing, automatic accurate floor-plan reconstruction, product-price scraping, 3D rendering, and unrestricted photo-library synchronization are outside the proposed scope.

7. Items to confirm before submission

Before submission, the team should approve this feature scope and responsibility split, insert the new Project 2 repository URL into the wiki/submission, and confirm with the instructor that explicit Google Photos Picker selection meets the intended photo-access requirement. The assignment is marked under construction and contains conflicting references to photos versus video, analytics point totals, and IE browser testing. We interpret the central media requirement as photos plus audio and will seek clarification for those inconsistencies. The linked presentation template also contains Cloud Vision slides absent from the main requirements; we will confirm whether those slides should be adapted to Gemini/ADK. Provider access and OAuth verification remain feasibility work, not completed features.

8. References

- Project 2 requirements

- Instructor's Speech Coach example and ADK architecture

- Instructor's Gemini/Node/Express guidance

- Instructor's Gemini versus Vertex AI discussion

- Analytics and logging requirements

- Peer-review process
- Existing Roomwise frontend
- Google Photos Picker guide
- Google Photos authorization
- Current Gemini SDK guidance
- Gemini audio understanding
- Google ADK TypeScript guidance
- Instructor presentation template
- Roomwise Stitch mockup project